

Encapsulamento



Sumário

Conceito

Justificativa

Métodos de acesso

Diferença para outras linguagens (C++)

Propriedades

Exemplos

Exercícios

Conceito

O que é encapsulamento?

- esconder detalhes internos de como algo funciona
- Expor apenas o necessário para que outros possam usar
- Proteger os dados de alterações indevidas

Uma cápsula de remédio: você não precisa saber sobre todos os compostos dela, apenas precisa saber que vai te curar



Exemplo: aumentar o volume de uma televisão

Qual circuito é acionado quando aumento o volume?

Qual algoritmo calcula o nível dos decibéis do volume?

Como os transistores processam o sinal?



Resposta: não interessa

Você só precisa saber “*este botão aumenta o volume da televisão*”, não como ele funciona por dentro

O funcionamento está **encapsulado** dentro do controle



Por que encapsular?

Segurança - Evita que dados sejam alterados de forma inadequada

```
class Conta:
    def __init__(self):
        self.saldo = 0

    def depositar(self, valor_deposito):
        self.saldo += valor_deposito

conta = Conta()
conta.saldo = 1000000 # ERRADO, qualquer um pode mudar o saldo

conta.depositar(1000) # CORRETO, só altera o saldo por operações
```


Manutenção - pode alterar o “interior” sem quebrar o código para quem usa

Se você mudar a forma como o depósito é feito, não altera a forma como o extrato é criado

Validação - Impede valores absurdos

```
class Pessoa:  
    def __init__(self):  
        self.idade = 0
```

```
pessoa = Pessoa()  
pessoa.idade = -5 #Faz sentido uma pessoa ter -5 anos?
```

Definição de visibilidade em C++

Uma conta bancária

```
1  class ContaBancaria {  
2      public:  
3          void depositar( const float valor );  
4          void sacar( const float valor);  
5          float consultarSaldo() const;  
6  
7      protected:  
8          int numeroConta;  
9  
10     private:  
11         float saldo;  
12     };
```

O que acontece se eu tentar acessar “saldo” diretamente?

```
5 ▶ int main() {  
6  
7     ContaBancaria conta = ContaBancaria( numeroConta: 123);  
8  
9     conta.saldo = 100000000;  
10 }  
11
```

O que acontece se eu tentar acessar “saldo” diretamente?

```
C:/workspace/Aprender e crescer/aprender-e-crescer/P00/aulas/06 - Encapsulamento/exemplos/visibilidade_cpp/main.cpp: In function 'int main()':
C:/workspace/Aprender e crescer/aprender-e-crescer/P00/aulas/06 - Encapsulamento/exemplos/visibilidade_cpp/main.cpp:13:11: error: 'float ContaBancaria::saldo' is private within this context
13 |     conta.saldo = 10000000;
    |     ^~~~~~
In file included from C:/workspace/Aprender e crescer/aprender-e-crescer/P00/aulas/06 - Encapsulamento/exemplos/visibilidade_cpp/main.cpp:3:
C:/workspace/Aprender e crescer/aprender-e-crescer/P00/aulas/06 - Encapsulamento/exemplos/visibilidade_cpp/src/contabancaria.h:13:11: note: declared private here
13 |     float saldo;
    |     ^~~~~~
ninja: build stopped: subcommand failed.
```

Convenções de visibilidade em Python

Funciona um pouco diferente...

Linguagem “*Consenting Adults*” (Adultos conscientes)

Não há bloqueio real como C++/Java

Tudo gira em torno de convenções

Como é definida a visibilidade em python

Público

sem underscore

Quando usar?

Métodos e atributos que podem ser conhecidos por todos

Protegido

um underscore

Quando usar?

Uso interno da classe e de suas “filhas”

Privado

dois underscores

Quando usar?

Dados sensíveis, que não deveriam ser conhecidos fora da classe

Público

```
class ContaBancaria:

    """
    construtor, depositar, sacar
    e consultar saldo
    sao metodo publicos (sem nenhum underscore)
    """

    def __init__(self, numeroConta):
        self.numero_conta = numeroConta
        self.saldo

    def depositar(self, valor):
        self.saldo += valor

    def sacar(self, valor):
        self.saldo -= valor

    def consultarSaldo(self):
        return self.saldo
```

Protegido (numero_conta)

```
def __init__(self, numeroConta):  
    """  
    meu numero da conta deve ser protegido  
    entao eu adiciona UM UNDERSCORE na frente  
    do seu nome  
    """  
    self._numero_conta = numeroConta  
    self.saldo
```

Protegido (tem saldo)

```
"""
quero poder verificar se tenho saldo
mas os outros lugares que usam essa classe

1 - nao precisam saber COMO eu valido
2 - nao precisam saber QUANTO exatamente eu tenho
"""

def _temSaldo(self):
    return self.__saldo > 0
```

Privado (saldo)

```
def __init__(self, numeroConta):  
    """  
    meu numero da conta deve ser protegido  
    entao eu adiciono UM UNDERSCORE na frente  
    do seu nome  
    """  
    self._numero_conta = numeroConta  
  
    """  
    meu saldo deve ser privado  
    entao um adiciono DOIS UNDERSCORES na frente  
    do seu nome  
  
    e altero os locais que usam ele  
    """  
    self.__saldo  
  
def depositar(self, valor):  
    self.__saldo += valor  
  
def sacar(self, valor):  
    self.__saldo -= valor  
  
def consultarSaldo(self):  
    return self.__saldo
```

```
class Exemplo:
    def __init__(self):
        self.publico = "Todo mundo vê"           # 🌐 Público
        self._protegido = "Uso interno"           # 🔒 Protegido
        self.__privado = "Bem escondido"          # 🔒 Privado

    def metodo_publico(self):                       # 🌐 Público
        pass

    def _metodo_protegido(self):                   # 🔒 Protegido
        pass

    def __metodo_privado(self):                   # 🔒 Privado
        pass
```

Métodos de acesso

Se meu atributo é privado, como eu manipulo ele quando preciso? Como pego o seu valor e o re-defino quando necessário?



SETTER

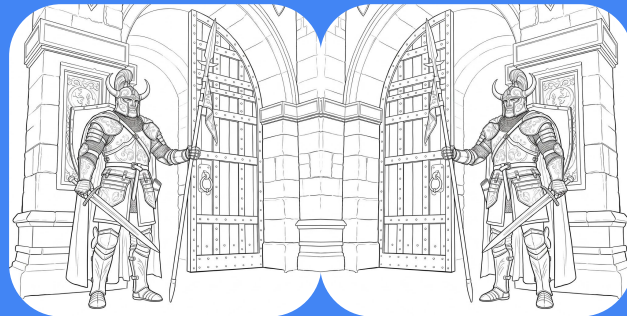
GETTER

O que eles resolvem?

```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

# Problema:
p = Pessoa("Carlos", 25)
p.idade = -10 # Aceita idade negativa!
p.idade = "vinte" # Aceita texto em vez de número!
```

A dupla dinâmica



Getter

Entrega o valor que você quer, mas pode formatar antes

Setter

Recebe o valor, mas pode validar ele antes de guardar em algum lugar

Como declarar e usar Getters e Setter (forma tradicional)

Spoiler: são apenas métodos públicos, que acessam atributos privados...

Getter

“get” -> pegar

get + propriedade

```
class Pessoa:
    def __init__(self, nome, idade):
        self._nome = nome
        self._idade = idade

    # GETTER - pega a idade
    def get_idade(self):
        return self._idade

    # GETTER - pega o nome
    def get_nome(self):
        return self._nome
```

Setter



Seether 
@SeetherOfficial
[Inscrever-se](#)



[▶ YouTube Mix](#)

Broken
435 mi de visualizações • há 16 anos
4:23

Fine Again
87 mi de visualizações • há 16 anos
4:05

Remedy
89 mi de visualizações • há 16 anos
3:28

Fake It
95 mi de visualizações
3:14

Setter

“set” -> definir

set + propriedade

```
# SETTER - define a idade (com validação)
def set_idade(self, valor):
    if not isinstance(valor, int):
        raise ValueError("Idade deve ser um número inteiro")

    if valor < 0 or valor > 150:
        raise ValueError("Idade deve estar entre 0 e 150")

    self._idade = valor

# SETTER - define o nome (com validação)
def set_nome(self, nome):
    if not isinstance(nome, str):
        raise ValueError("Nome deve ser um texto")

    if len(nome) == 0:
        raise ValueError("Nome não pode ser vazio")

    self._nome = nome
```

Como usar?

```
p = Pessoa("Carlos", 25)
print(p.get_idade()) # O que sai aqui?
p.set_idade(30) # OK
p.set_idade(-5) # O que acontece aqui?
p.set_nome("") # O que acontece aqui?
p.set_nome(123) # O que acontece aqui?
p.set_nome("Ana")
print(p.get_nome()) # o que sai aqui?
```

Que outro lugar eu preciso de
uma validação antes de fazer
alguma coisa?

Conta bancária

Faz sentido eu sacar um valor que não tenho na conta?

O que aconteceria se eu controlasse diretamente a variável *saldo*?

```
class ContaBancaria:
    def __init__(self, numeroConta):
        self._numero_conta = numeroConta
        self.__saldo = 0

    def depositar(self, valor):
        self.__saldo += valor

    def sacar(self, valor_a_sacar):
        if self.consultarSaldo() < valor_a_sacar:
            raise ValueError("Saldo insuficiente")

        self.__saldo -= valor_a_sacar

    def consultarSaldo(self):
        return self.__saldo
```

Jeito *Pythoniano* (@property)

Como usar?

- defina um método com o mesmo nome da propriedade que deseja acessar
- retorne o atributo privado
- decore com @property

```
@property
def idade(self):
    print("chamando getter")
    return self.__idade
```

Como chamar?

```
print(p1.idade)
```

```
# Verifica a idade após a tentativa de atribuição
```

Mas...

Isso não é a mesma coisa que fizemos até agora, sem encapsular?

Sim, e é exatamente essa a ideia do decorator *@property*, deixar tudo do jeito mais fácil, *sintaxicamente* falando

Exemplo

E o setter?

É um pouco diferente...

ao invés de *@property*

@propriedade.setter

```
@idade.setter
def idade(self, valor):
    print("chamando setter")
    if isinstance(valor, int) and valor >= 0:
        self.__idade = valor
    else:
        print(
            "Idade inválida! Deve ser um número inteiro não negativo.")
```


Como usar

- crie um método com o mesmo nome da propriedade
 - receba o novo valor por parâmetro
- faça as validações necessárias
- decore com `@propriedade.setter`

```
@idade.setter
def idade(self, valor):
    print("chamando setter")
    if isinstance(valor, int) and valor >= 0:
        self.__idade = valor
    else:
        print(
            "Idade inválida! Deve ser um número inteiro não negativo.")
```

Encapsulamento



Exercícios